Lab Experiment 7: CI/CD using Jenkins, GitHub and Docker Hub

1. Aim

To design and implement a complete CI/CD pipeline using **Jenkins**, integrating source code from **GitHub**, and building & pushing Docker images to **Docker Hub**.

2. Objectives

- Understand CI/CD workflow using Jenkins (GUI-based tool)
- Create a structured GitHub repository with application + Jenkinsfile
- Build Docker images from source code
- Securely store Docker Hub credentials in Jenkins
- Automate build & push process using webhook triggers
- Use same host (Docker) as Jenkins agent

3. Theory

What is Jenkins?

Jenkins is a **web-based GUI automation server** used to:

- Build applications
- Test code
- Deploy software

It provides:

- Dashboard (browser-based UI)
- Plugin ecosystem (GitHub, Docker, etc.)
- Pipeline as Code using `Jenkinsfile`

What is CI/CD?

- **Continuous Integration (CI):**

Code is automatically built and tested after each commit

- **Continuous Deployment (CD):**

Built artifacts (Docker images) are automatically delivered/deployed

Workflow Overview

```
Developer → GitHub → Webhook → Jenkins → Build → Docker Hub
```

4. Prerequisites

- Docker & Docker Compose installed
- GitHub account
- Docker Hub account
- Basic Linux command knowledge

5. Part A: GitHub Repository Setup (Source Code + Build Definition)

5.1 Create Repository

Create a repository on GitHub:

```
my-app
```

5.2 Project Structure

```
my-app/
├── app.py
├── requirements.txt
├── Dockerfile
├── Jenkinsfile
```

5.3 Application Code

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from CI/CD Pipeline!"
    #return "Hello from CI/CD Pipeline!, my sapid is 123456"

app.run(host="0.0.0.0", port=80)
```

requirements.txt

```
flask
```

5.4 Dockerfile (Build Process)

```
FROM python:3.10-slim

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80
CMD ["python", "app.py"]
```

Build Process Explanation

1. Source code pushed to GitHub
2. Jenkins pulls code
3. Dockerfile:
    - Creates environment
    - Installs dependencies
    - Packages app
4. Output → Docker Image

5.5 Jenkinsfile (Pipeline Definition in GitHub)

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "your-dockerhub-username/myapp"
    }

    stages {

        stage('Clone Source') {
            steps {
                git 'https://github.com/your-username/my-app.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]
                    sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --password-stdi
                }
            }
        }
```

```
        stage('Push to Docker Hub') {
            steps {
                sh 'docker push $IMAGE_NAME:latest'
            }
        }
    }
}
```


6. Part B: Jenkins Setup using Docker (Persistent Configuration)

6.1 Create Docker Compose File

```yaml
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
```

6.2 Start Jenkins

```
docker-compose up -d
```

Access:

```
http://localhost:8080
```

6.3 Unlock Jenkins

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

6.4 Initial Setup

- Install suggested plugins
- Create admin user

7. Part C: Jenkins Configuration

7.1 Add Docker Hub Credentials

Path:

```
Manage Jenkins → Credentials → Add Credentials
```

- Type: Secret Text
- ID: `dockerhub-token`
- Value: Docker Hub Access Token

7.2 Create Pipeline Job

1. New Item → Pipeline
2. Name: `ci-cd-pipeline`

Configure:

```
Pipeline script from SCM
```

- SCM: Git
- Repo URL: your GitHub repo
- Script Path: `Jenkinsfile`

8. Part D: GitHub Webhook Integration

8.1 Configure Webhook

In GitHub:

```
Settings → Webhooks → Add Webhook
```

Payload URL:

```
http://<your-server-ip>:8080/github-webhook/
```

Events:

```
Push events
```


9. Part E: Execution Flow (Stage-wise Explanation)

Stage 1: Code Push

- Developer updates code in GitHub

Stage 2: Webhook Trigger

- GitHub sends event to Jenkins

Stage 3: Jenkins Pipeline Execution

Stage: Clone

- Pulls latest code from GitHub

Stage: Build

- Docker builds image using Dockerfile

Stage: Auth

- Jenkins logs into Docker Hub using stored token

Stage: Push

- Image pushed to Docker Hub

Stage 4: Artifact Ready

- Docker image available globally

10. Role of Same Host Agent

- Jenkins runs inside Docker
- Docker socket mounted:

```
/var/run/docker.sock
```

Effect

- Jenkins directly controls host Docker
- Builds and pushes images without separate agent

11. Observations

- Jenkins GUI simplifies CI/CD management
- GitHub acts as source + pipeline definition
- Docker ensures consistent builds
- Webhook enables automation

12. Result

Successfully implemented a complete CI/CD pipeline where:

- Source code and pipeline are maintained in GitHub
- Jenkins automatically detects changes
- Docker image is built on host agent
- Image is securely pushed to Docker Hub

13. Viva Questions

1. What is the role of Jenkinsfile?
2. How does Jenkins integrate with GitHub?
3. Why is Docker used in CI/CD?
4. What is a webhook?
5. Why store Docker Hub token in Jenkins credentials?
6. What is the benefit of using same host as agent?

14. Key Notes

- Jenkins is **GUI-based but pipeline is code-driven**
- Always use **credentials store (never hardcode secrets)**
- Webhook makes CI/CD fully automatic
- This setup is ideal for **learning and small deployments**

Understanding Jenkins Pipeline Syntax (Simplified Explanation)

Jenkins pipelines (written in a `Jenkinsfile`) look confusing at first, but they follow a **clear structure of blocks and steps**.

1. Basic Pipeline Structure

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                sh 'echo Hello'
            }
        }
    }
}
```

2. Key Terms Explained

1. `pipeline {}`

- Root block
- Everything is written inside this

2. `agent`

```
agent any
```

- Defines **where the pipeline runs**
- `any` → run on any available node (in your case, same Docker host)

3. `stages {}`

- Groups all phases of pipeline

- Logical separation (Clone, Build, Test, Deploy)

4. `stage('Name')`

```
stage('Build')
```

- A **single step/phase** in pipeline
- Visible in Jenkins GUI as blocks

5. `steps {}`

- Contains actual commands to execute

6. Common Steps

`git`

```
git 'https://github.com/user/repo.git'
```

- Clones source code from **GitHub**

`sh` **(Shell Command)**

```
sh 'echo Hello'
```

- Runs Linux commands inside agent
- Equivalent to terminal command

`echo`

```
echo "Build started"
```

- Prints message in Jenkins console log

3. Example (Readable Version)

```
pipeline {
    agent any

    stages {

        stage('Clone') {
            steps {
                git 'https://github.com/user/repo.git'
            }
        }

        stage('Build') {
            steps {
                echo "Building..."
                sh 'docker build -t myapp .'
            }
        }

        stage('Deploy') {
            steps {
                sh 'docker run -d myapp'
            }
        }
    }
}
```


*4. The Challenging Part: `withCredentials` *

This is a difficult part for beginner.

Original Code

```
withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')])
    sh 'echo $DOCKER_TOKEN | docker login -u your-username --password-stdin'
}
```

5. What Problem Does This Solve?

You need to login to **Docker Hub**, but:

You should NOT write password directly in code Jenkins stores it securely

6. Step-by-Step Meaning

Step 1: Store Secret in Jenkins

- You saved token with ID:

```
dockerhub-token
```

Step 2: `withCredentials` Block

```
withCredentials([...]) {
    // use secret here
}
```

- Temporarily **injects secret into environment variable**

Step 3: Inside the Bracket

```
string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')
```

Meaning:

Part	Meaning
`string`	Type of secret (text token)
`credentialsId`	Name used in Jenkins
`variable`	Temporary variable name

Step 4: What Happens Internally

Jenkins does:

```
DOCKER_TOKEN=your_actual_secret_value
```

Step 5: Using It in `sh`

```
sh 'echo $DOCKER_TOKEN | docker login -u username --password-stdin'
```

- `$DOCKER_TOKEN` → replaced with actual token
- `--password-stdin` → secure login (no plain password)

7. Simplified Analogy

Think of it like:

```
Locker (Jenkins Credentials)
   ↓
You ask by ID (dockerhub-token)
   ↓
Jenkins gives temporary key (DOCKER_TOKEN)
```

```
   ↓
You use it → then it disappears
```

11/12

8. Simplified Version (Mentally Understand Like This)

```
withCredentials(...) {
    // Jenkins gives you secret temporarily
    use secret here
}
```

9. Common Mistakes

Writing password directly:

```
sh 'docker login -u user -p mypassword'
```

Wrong credentialsId:

```
credentialsId: 'wrong-name'
```

Using variable outside block:

```
echo $DOCKER_TOKEN   //  won't work outside
```

10. Clean Final Example (Student Friendly)

```
stage('Push to Docker Hub') {
    steps {
        withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {

            echo "Logging into Docker Hub"

            sh '''
            echo $DOCKER_TOKEN | docker login -u your-username --password-stdin
            docker push your-username/myapp:latest
            '''
        }
    }
}
```

12/12

11. Key Takeaways

- `pipeline → stages → stage → steps` = structure
- `sh` = run commands
- `git` = fetch code
- `withCredentials` = securely use secrets
- Secrets are **temporary and protected**